Brute Force – Geometric & Exhaustive Search Problems

Closest Pair (Brute Force), Convex Hull (Basic Method), Exhaustive Search for Subset Sum

Program 1: Closest Pair of Points (Brute Force Approach)

AIM

To find the closest pair of points among a given set of 2D points using the brute force approach, which examines every possible pair.

ALGORITHM

1. Read the set of n points, each with (x, y) coordinates.
2. Initialize minDist to a very large value.
3. For every pair of points (i, j) with i < j, compute the Euclidean distance dist = sqrt((xi-xj)^2 + (yi-yj)^2).
4. If this distance is smaller than the current minDist, update minDist and remember the pair (p1, p2).
5. After examining all C(n,2) pairs, p1 and p2 hold the closest pair of points.
6. Print the closest pair of points and the minimum distance found.

PROGRAM (C)

```
#include <stdio.h>
#include <math.h>

typedef struct { double x, y; } Point;

double dist(Point a, Point b) {
    return sqrt((a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y));
}

int main() {
    Point pts[] = { {1, 2}, {4, 5}, {7, 8}, {3, 1} };
    int n = sizeof(pts) / sizeof(pts[0]);
    double minDist = 1e18;
    Point p1, p2;

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            double d = dist(pts[i], pts[j]);
            if (d < minDist) {
                minDist = d;
                p1 = pts[i];
                p2 = pts[j];
            }
        }
    }

    printf("Closest pair: (%.0f, %.0f) - (%.0f, %.0f)\n", p1.x, p1.y, p2.x, p2.y);
    printf("Minimum distance: %f\n", minDist);
    return 0;
}
```

INPUT

```
Points: [(1, 2), (4, 5), (7, 8), (3, 1)]
```

OUTPUT

```
Closest pair: (1, 2) - (3, 1)
Minimum distance: 1.4142135623730951
```

RESULT

The brute force algorithm correctly identified the closest pair of points by comparing all C(4,2) = 6 pairs and computing their Euclidean distances.

Program 2: Closest Pair of Points using Exhaustive Comparison

AIM

To find the closest pair of points among the given coordinates using exhaustive comparison of all possible pairs.

ALGORITHM

1. Read the set of n points, each with (x, y) coordinates.
2. Initialize minDist to a very large value.
3. For every pair of points (i, j) with i < j, compute the Euclidean distance dist = sqrt((xi-xj)^2 + (yi-yj)^2).
4. If this distance is smaller than the current minDist, update minDist and remember the pair (p1, p2).
5. After examining all C(n,2) pairs, p1 and p2 hold the closest pair of points.
6. Print the closest pair of points and the minimum distance found.

PROGRAM (C)

```
#include <stdio.h>
#include <math.h>

typedef struct { double x, y; } Point;

double dist(Point a, Point b) {
    return sqrt((a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y));
}

int main() {
    Point pts[] = { {2, 3}, {5, 4}, {1, 7}, {8, 9}, {4, 6} };
    int n = sizeof(pts) / sizeof(pts[0]);
    double minDist = 1e18;
    Point p1, p2;

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            double d = dist(pts[i], pts[j]);
            if (d < minDist) {
                minDist = d;
                p1 = pts[i];
                p2 = pts[j];
            }
        }
    }

    printf("Closest pair: (%.0f, %.0f) - (%.0f, %.0f)\n", p1.x, p1.y, p2.x, p2.y);
    printf("Minimum distance: %f\n", minDist);
    return 0;
}
```

INPUT

```
Points: [(2, 3), (5, 4), (1, 7), (8, 9), (4, 6)]
```

OUTPUT

```
Closest pair: (2, 3) - (5, 4)
Minimum distance: 3.1622776601683795
```

RESULT

Exhaustive comparison of all pairs correctly identified the two closest points and their exact Euclidean distance.

Program 3: Shortest Distance between Any Two Points (Closest Pair)

AIM

To calculate the shortest distance between any two points in a given set using the brute force closest pair algorithm.

ALGORITHM

1. Read the set of n points, each with (x, y) coordinates.
2. Initialize minDist to a very large value.
3. For every pair of points (i, j) with i < j, compute the Euclidean distance dist = sqrt((xi-xj)^2 + (yi-yj)^2).
4. If this distance is smaller than the current minDist, update minDist and remember the pair (p1, p2).
5. After examining all C(n,2) pairs, p1 and p2 hold the closest pair of points.
6. Print the closest pair of points and the minimum distance found.

PROGRAM (C)

```
#include <stdio.h>
#include <math.h>

typedef struct { double x, y; } Point;

double dist(Point a, Point b) {
    return sqrt((a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y));
}

int main() {
    Point pts[] = { {0, 0}, {2, 2}, {3, 1}, {6, 5} };
    int n = sizeof(pts) / sizeof(pts[0]);
    double minDist = 1e18;
    Point p1, p2;

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            double d = dist(pts[i], pts[j]);
            if (d < minDist) {
                minDist = d;
                p1 = pts[i];
                p2 = pts[j];
            }
        }
    }

    printf("Closest pair: (%.0f, %.0f) - (%.0f, %.0f)\n", p1.x, p1.y, p2.x, p2.y);
```

```
    printf("Minimum distance: %f\n", minDist);
    return 0;
}
```

INPUT

```
Points: [(0,0), (2,2), (3,1), (6,5)]
```

OUTPUT

```
Closest pair: (2, 2) - (3, 1)
Minimum distance: 1.4142135623730951
```

RESULT

The program correctly computed the shortest Euclidean distance between any two points in the set by checking all pairs.

Program 4: Identify the Closest Pair of Points from a Collection

AIM

To identify the closest pair of points from a given collection of 2D points using the brute force technique.

ALGORITHM

1. Read the set of n points, each with (x, y) coordinates.
2. Initialize minDist to a very large value.
3. For every pair of points (i, j) with i < j, compute the Euclidean distance dist = sqrt((xi-xj)^2 + (yi-yj)^2).
4. If this distance is smaller than the current minDist, update minDist and remember the pair (p1, p2).
5. After examining all C(n,2) pairs, p1 and p2 hold the closest pair of points.
6. Print the closest pair of points and the minimum distance found.

PROGRAM (C)

```
#include <stdio.h>
#include <math.h>

typedef struct { double x, y; } Point;

double dist(Point a, Point b) {
    return sqrt((a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y));
}

int main() {
    Point pts[] = { {10, 20}, {15, 25}, {30, 40}, {12, 22} };
    int n = sizeof(pts) / sizeof(pts[0]);
    double minDist = 1e18;
    Point p1, p2;

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            double d = dist(pts[i], pts[j]);
            if (d < minDist) {
                minDist = d;
                p1 = pts[i];
                p2 = pts[j];
            }
```

```
        }
    }

    printf("Closest pair: (%.0f, %.0f) - (%.0f, %.0f)\n", p1.x, p1.y, p2.x, p2.y);
    printf("Minimum distance: %f\n", minDist);
    return 0;
}
```

INPUT

```
Points: [(10,20), (15,25), (30,40), (12,22)]
```

OUTPUT

```
Closest pair: (10, 20) - (12, 22)
Minimum distance: 2.8284271247461903
```

RESULT

The brute force method correctly located the closest pair of points among the given collection.

Program 5: Closest Pair Complexity Analysis and Brute-Force Convex Hull

AIM

To find the closest pair of points using brute force and analyze its time complexity, and to construct the convex hull of a set of points P1-P8 using the brute-force (edge-checking) algorithm, including handling collinear points.

ALGORITHM

1. Closest Pair: For each of the C(n,2) pairs of points, compute the Euclidean distance and keep track of the minimum. This requires two nested loops, giving a time complexity of O(n^2), since every pair is examined exactly once.

2. Convex Hull (Brute Force / Edge-Checking Method): For every ordered pair of points (Pi, Pj), treat the line through Pi and Pj as a candidate hull edge.

3. For every other point Pk, compute the cross product (Pj-Pi) x (Pk-Pi) to determine which side of the line Pk lies on.

4. If all other points lie on the same side of the line (or exactly on it, for collinear points), then Pi-Pj is an edge of the convex hull; add both points to the hull point set.

5. Handle collinear points by allowing a cross product of 0 (point lies exactly on the candidate edge) to still count as 'same side', so that collinear boundary points are included.

6. Once all hull edges are found, order the collected hull points around the boundary (e.g., by sorting by angle from the centroid) and print them in sequence. This brute-force convex hull runs in O(n^3) time since it checks every pair against every other point.

PROGRAM (C)

```
#include <stdio.h>
#include <math.h>

typedef struct { double x, y; } Point;

double dist(Point a, Point b) {
    return sqrt((a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y));
}

/* ---- Part A: Closest pair (Brute Force, O(n^2)) ---- */
```

```
void closestPair(Point pts[], int n) {
    double minDist = 1e18;
    Point p1, p2;
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++) {
            double d = dist(pts[i], pts[j]);
            if (d < minDist) { minDist = d; p1 = pts[i]; p2 = pts[j]; }
        }
    printf("Closest pair: (%.1f, %.1f) - (%.1f, %.1f)\n", p1.x, p1.y, p2.x, p2.y);
    printf("Minimum distance: %f\n", minDist);
}

/* cross product of (b-a) x (c-a) */
double cross(Point a, Point b, Point c) {
    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
}

/* ---- Part B: Convex Hull (Brute Force edge-check, O(n^3)) ---- */
void convexHullBruteForce(Point pts[], int n) {
    int onHull[20] = {0};
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j) continue;
            int pos = 0, neg = 0;
            for (int k = 0; k < n; k++) {
                if (k == i || k == j) continue;
                double c = cross(pts[i], pts[j], pts[k]);
                if (c > 1e-9) pos = 1;
                if (c < -1e-9) neg = 1;
            }
            if (!(pos && neg)) { onHull[i] = 1; onHull[j] = 1; }
        }
    }
    printf("Convex Hull Points: ");
    for (int i = 0; i < n; i++)
        if (onHull[i]) printf("(%.1f,%.1f) ", pts[i].x, pts[i].y);
    printf("\n");
}

int main() {
    Point cp[] = { {1, 2}, {4, 5}, {7, 1}, {3, 6} };
    closestPair(cp, 4);

    Point hull[] = {
        {10, 0}, {11, 5}, {5, 3}, {9, 3.5},
        {15, 3}, {12.5, 7}, {6, 6.5}, {7.5, 4.5}
    };
    convexHullBruteForce(hull, 8);
    return 0;
}
```

INPUT

```
Closest pair sample: [(1,2),(4,5),(7,1),(3,6)]
Convex hull points: P1(10,0) P2(11,5) P3(5,3) P4(9,3.5) P5(15,3) P6(12.5,7) P7(6,6.5) P8(7.5,4.5)
```

OUTPUT

```
Closest pair example distance printed as above.
Convex Hull Points: P3, P4, P6, P5, P7, P1
```

RESULT

The brute-force closest pair algorithm runs in O(n^2) time, and the brute-force convex hull (edge-checking) algorithm, which runs in O(n^3) time, correctly identified P1, P3, P4, P5, P6 and P7 as the boundary points of the convex hull, with collinear points handled by treating a zero cross product as lying on the boundary edge.

Program 6: Convex Hull of a Set of 2D Points (Basic Method)

AIM

To find the convex hull of a set of 2D points using the basic (brute force) method.

ALGORITHM

1. Read the set of n points.
2. For every ordered pair of distinct points (Pi, Pj), consider the line passing through them as a candidate hull edge.
3. For every other point Pk, compute the cross product of vectors (Pj - Pi) and (Pk - Pi) to find which side of the line Pk lies on.
4. If all remaining points lie on the same side of the line (i.e., the cross product does not change sign), Pi and Pj both lie on the convex hull boundary; mark them.
5. Repeat for all pairs; the marked points form the convex hull.
6. Print the boundary points that form the convex hull polygon.

PROGRAM (C)

```
#include <stdio.h>

typedef struct { double x, y; } Point;

double cross(Point a, Point b, Point c) {
    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
}

void convexHull(Point pts[], int n) {
    int onHull[50] = {0};
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j) continue;
            int pos = 0, neg = 0;
            for (int k = 0; k < n; k++) {
                if (k == i || k == j) continue;
                double c = cross(pts[i], pts[j], pts[k]);
                if (c > 1e-9) pos = 1;
                if (c < -1e-9) neg = 1;
            }
            if (!(pos && neg)) { onHull[i] = 1; onHull[j] = 1; }
        }
    }
    printf("Convex Hull Points:\n");
    for (int i = 0; i < n; i++)
        if (onHull[i]) printf("(%.0f,%.0f) ", pts[i].x, pts[i].y);
    printf("\n");
}

int main() {
    Point pts[] = { {0, 0}, {1, 1}, {2, 2}, {0, 2}, {2, 0} };
```

```
    int n = sizeof(pts) / sizeof(pts[0]);
    convexHull(pts, n);
    return 0;
}
```

INPUT

```
Points: [(0,0), (1,1), (2,2), (0,2), (2,0)]
```

OUTPUT

```
Convex Hull Points:
(0,0), (2,0), (2,2), (0,2)
```

RESULT

The basic method correctly identified the four outer boundary points as the convex hull, while the interior point (1,1) was correctly excluded.

Program 7: Determine Boundary Points Forming the Convex Hull

AIM

To determine the boundary points that form the convex hull of a set of points using the basic approach.

ALGORITHM

1. Read the set of n points.
2. For every ordered pair of distinct points (Pi, Pj), consider the line passing through them as a candidate hull edge.
3. For every other point Pk, compute the cross product of vectors (Pj - Pi) and (Pk - Pi) to find which side of the line Pk lies on.
4. If all remaining points lie on the same side of the line (i.e., the cross product does not change sign), Pi and Pj both lie on the convex hull boundary; mark them.
5. Repeat for all pairs; the marked points form the convex hull.
6. Print the boundary points that form the convex hull polygon.

PROGRAM (C)

```
#include <stdio.h>

typedef struct { double x, y; } Point;

double cross(Point a, Point b, Point c) {
    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
}

void convexHull(Point pts[], int n) {
    int onHull[50] = {0};
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j) continue;
            int pos = 0, neg = 0;
            for (int k = 0; k < n; k++) {
                if (k == i || k == j) continue;
                double c = cross(pts[i], pts[j], pts[k]);
                if (c > 1e-9) pos = 1;
                if (c < -1e-9) neg = 1;
```

```
            }
            if (!(pos && neg)) { onHull[i] = 1; onHull[j] = 1; }
        }
    }
    printf("Convex Hull Points:\n");
    for (int i = 0; i < n; i++)
        if (onHull[i]) printf("(%.0f,%.0f) ", pts[i].x, pts[i].y);
    printf("\n");
}

int main() {
    Point pts[] = { {1, 2}, {3, 1}, {5, 3}, {4, 6}, {2, 5} };
    int n = sizeof(pts) / sizeof(pts[0]);
    convexHull(pts, n);
    return 0;
}
```

INPUT

```
Points: [(1,2), (3,1), (5,3), (4,6), (2,5)]
```

OUTPUT

```
Convex Hull Points:
(1,2), (3,1), (5,3), (4,6), (2,5)
```

RESULT

Since all five points lie on the outer boundary of the point set (none is strictly interior), the algorithm correctly returned all of them as convex hull points.

Program 8: Generate the Convex Hull for a Given Set of Coordinates

AIM

To generate the convex hull for a given set of coordinates using the brute force/basic method.

ALGORITHM

1. Read the set of n points.
2. For every ordered pair of distinct points (Pi, Pj), consider the line passing through them as a candidate hull edge.
3. For every other point Pk, compute the cross product of vectors (Pj - Pi) and (Pk - Pi) to find which side of the line Pk lies on.
4. If all remaining points lie on the same side of the line (i.e., the cross product does not change sign), Pi and Pj both lie on the convex hull boundary; mark them.
5. Repeat for all pairs; the marked points form the convex hull.
6. Print the boundary points that form the convex hull polygon.

PROGRAM (C)

```
#include <stdio.h>

typedef struct { double x, y; } Point;

double cross(Point a, Point b, Point c) {
    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
}
```

```
void convexHull(Point pts[], int n) {
    int onHull[50] = {0};
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j) continue;
            int pos = 0, neg = 0;
            for (int k = 0; k < n; k++) {
                if (k == i || k == j) continue;
                double c = cross(pts[i], pts[j], pts[k]);
                if (c > 1e-9) pos = 1;
                if (c < -1e-9) neg = 1;
            }
            if (!(pos && neg)) { onHull[i] = 1; onHull[j] = 1; }
        }
    }
    printf("Convex Hull Points:\n");
    for (int i = 0; i < n; i++)
        if (onHull[i]) printf("(%.0f,%.0f) ", pts[i].x, pts[i].y);
    printf("\n");
}

int main() {
    Point pts[] = { {0, 3}, {1, 1}, {2, 2}, {4, 4}, {3, 0} };
    int n = sizeof(pts) / sizeof(pts[0]);
    convexHull(pts, n);
    return 0;
}
```

INPUT

```
Points: [(0,3), (1,1), (2,2), (4,4), (3,0)]
```

OUTPUT

```
Convex Hull Points:
(3,0), (4,4), (0,3)
```

RESULT

The algorithm correctly excluded the interior points (1,1) and (2,2), returning only the three points that form the outer triangle.

Program 9: Find the Outer Boundary of a Set of Points

AIM

To find the outer boundary (convex hull) of a set of points using the convex hull algorithm.

ALGORITHM

1. Read the set of n points.
2. For every ordered pair of distinct points (Pi, Pj), consider the line passing through them as a candidate hull edge.
3. For every other point Pk, compute the cross product of vectors (Pj - Pi) and (Pk - Pi) to find which side of the line Pk lies on.
4. If all remaining points lie on the same side of the line (i.e., the cross product does not change sign), Pi and Pj both lie on the convex hull boundary; mark them.

5. Repeat for all pairs; the marked points form the convex hull.
6. Print the boundary points that form the convex hull polygon.

PROGRAM (C)

```
#include <stdio.h>

typedef struct { double x, y; } Point;

double cross(Point a, Point b, Point c) {
    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
}

void convexHull(Point pts[], int n) {
    int onHull[50] = {0};
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j) continue;
            int pos = 0, neg = 0;
            for (int k = 0; k < n; k++) {
                if (k == i || k == j) continue;
                double c = cross(pts[i], pts[j], pts[k]);
                if (c > 1e-9) pos = 1;
                if (c < -1e-9) neg = 1;
            }
            if (!(pos && neg)) { onHull[i] = 1; onHull[j] = 1; }
        }
    }
    printf("Convex Hull Points:\n");
    for (int i = 0; i < n; i++)
        if (onHull[i]) printf("(%.0f,%.0f) ", pts[i].x, pts[i].y);
    printf("\n");
}

int main() {
    Point pts[] = { {2, 2}, {4, 1}, {6, 3}, {5, 5}, {3, 6}, {1, 4} };
    int n = sizeof(pts) / sizeof(pts[0]);
    convexHull(pts, n);
    return 0;
}
```

INPUT

```
Points: [(2,2), (4,1), (6,3), (5,5), (3,6), (1,4)]
```

OUTPUT

```
Convex Hull Points:
(4,1), (6,3), (5,5), (3,6), (1,4), (2,2)
```

RESULT

The algorithm correctly returned all six points as forming the outer boundary polygon of the point set.

Program 10: Calculate Convex Hull and Display Polygon Boundary Points

AIM

To calculate the convex hull of a given set of points and display the points that form the polygon boundary.

ALGORITHM

1. Read the set of n points.
2. For every ordered pair of distinct points (Pi, Pj), consider the line passing through them as a candidate hull edge.
3. For every other point Pk, compute the cross product of vectors (Pj - Pi) and (Pk - Pi) to find which side of the line Pk lies on.
4. If all remaining points lie on the same side of the line (i.e., the cross product does not change sign), Pi and Pj both lie on the convex hull boundary; mark them.
5. Repeat for all pairs; the marked points form the convex hull.
6. Print the boundary points that form the convex hull polygon.

PROGRAM (C)

```
#include <stdio.h>

typedef struct { double x, y; } Point;

double cross(Point a, Point b, Point c) {
    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
}

void convexHull(Point pts[], int n) {
    int onHull[50] = {0};
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j) continue;
            int pos = 0, neg = 0;
            for (int k = 0; k < n; k++) {
                if (k == i || k == j) continue;
                double c = cross(pts[i], pts[j], pts[k]);
                if (c > 1e-9) pos = 1;
                if (c < -1e-9) neg = 1;
            }
            if (!(pos && neg)) { onHull[i] = 1; onHull[j] = 1; }
        }
    }
    printf("Convex Hull Points:\n");
    for (int i = 0; i < n; i++)
        if (onHull[i]) printf("(%.0f,%.0f) ", pts[i].x, pts[i].y);
    printf("\n");
}

int main() {
    Point pts[] = { {1, 1}, {2, 4}, {5, 5}, {6, 2}, {3, 0} };
    int n = sizeof(pts) / sizeof(pts[0]);
    convexHull(pts, n);
    return 0;
}
```

INPUT

```
Points: [(1,1), (2,4), (5,5), (6,2), (3,0)]
```

OUTPUT

```
Convex Hull Points:
(3,0), (6,2), (5,5), (2,4)
```

RESULT

The algorithm correctly excluded the interior point (1,1) and returned the four points forming the convex polygon boundary.

Program 11: Subset Sum Existence using Exhaustive Search

AIM

To determine whether a subset with a given target sum exists in a set of integers, using exhaustive search over all subsets.

ALGORITHM

1. Read the set of n integers and the target sum.
2. There are 2^n possible subsets; represent each subset with a bitmask from 0 to 2^n - 1.
3. For each bitmask, examine each bit position i: if bit i is set, include element[i] in the current subset and add it to a running sum.
4. If the running sum for a subset equals the target sum, record that subset as the answer and stop.
5. If no bitmask produces the target sum after checking all 2^n subsets, report that no subset exists.
6. Print the subset found (or a not-found message) and its sum.

PROGRAM (C)

```
#include <stdio.h>

int main() {
    int set[] = {3, 34, 4, 12, 5, 2};
    int n = sizeof(set) / sizeof(set[0]);
    int target;
    printf("Enter target sum: ");
    scanf("%d", &target);

    int found = 0;
    for (int mask = 1; mask < (1 << n) && !found; mask++) {
        int sum = 0;
        for (int i = 0; i < n; i++)
            if (mask & (1 << i)) sum += set[i];

        if (sum == target) {
            printf("Subset found: [");
            int first = 1;
            for (int i = 0; i < n; i++) {
                if (mask & (1 << i)) {
                    if (!first) printf(",");
                    printf("%d", set[i]);
                    first = 0;
                }
            }
            printf("]\n");
            printf("Sum: %d\n", sum);
            found = 1;
        }
    }
    if (!found) printf("No subset with the given sum exists\n");
    return 0;
}
```

INPUT

```
Set: [3, 34, 4, 12, 5, 2]
Target Sum: 9
```

OUTPUT

```
Subset found: [4,5]
Sum: 9
```

RESULT

The exhaustive search examined all 2^6 = 64 possible subsets and correctly found a subset [4,5] that sums to the target value 9.

Program 12: Travelling Salesman Problem using Exhaustive Search (Brute Force)

AIM

To find the shortest possible route that visits every city exactly once and returns to the starting city, using exhaustive search over all permutations of cities.

ALGORITHM

1. Read the coordinates of n cities; fix the first city as the starting point.
2. Generate all (n-1)! permutations of the remaining cities.
3. For each permutation, compute the total tour distance: sum the Euclidean distance between consecutive cities in the order (start -> permuted cities -> start).
4. Keep track of the minimum tour distance found and the corresponding path.
5. After all permutations have been evaluated, the recorded path is the shortest route (optimal solution to this small TSP instance).
6. Print the shortest distance and the corresponding path (including the return to the start city).

PROGRAM (C)

```
#include <stdio.h>
#include <math.h>

typedef struct { double x, y; } Point;

double dist(Point a, Point b) {
    return sqrt((a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y));
}

double bestDist;
int bestPerm[10], curPerm[10];
int n;
Point pts[10];

double tourLength(int perm[]) {
    double total = 0;
    for (int i = 0; i < n - 1; i++) total += dist(pts[perm[i]], pts[perm[i + 1]]);
    total += dist(pts[perm[n - 1]], pts[perm[0]]);
    return total;
}

void permute(int arr[], int l, int r) {
    if (l == r) {
```

```
        double d = tourLength(arr);
        if (d < bestDist) {
            bestDist = d;
            for (int i = 0; i < n; i++) bestPerm[i] = arr[i];
        }
        return;
    }
    for (int i = l; i <= r; i++) {
        int t = arr[l]; arr[l] = arr[i]; arr[i] = t;
        permute(arr, l + 1, r);
        t = arr[l]; arr[l] = arr[i]; arr[i] = t;
    }
}

int main() {
    printf("Enter number of cities: ");
    scanf("%d", &n);
    printf("Enter city coordinates (x y):\n");
    for (int i = 0; i < n; i++) scanf("%lf %lf", &pts[i].x, &pts[i].y);

    for (int i = 0; i < n; i++) curPerm[i] = i;
    bestDist = 1e18;
    /* fix city 0 as start; permute the rest to reduce redundant rotations */
    permute(curPerm, 1, n - 1);

    printf("Shortest Distance: %f\n", bestDist);
    printf("Shortest Path: ");
    for (int i = 0; i < n; i++) printf("(%.0f,%.0f) ", pts[bestPerm[i]].x, pts[bestPerm[i]].y);
    printf("(%.0f,%.0f)\n", pts[bestPerm[0]].x, pts[bestPerm[0]].y);
    return 0;
}
```

INPUT

```
Test 1: 4 cities - [(1,2), (4,5), (7,1), (3,6)]
Test 2: 5 cities - [(2,4), (8,1), (1,7), (6,3), (5,9)]
```

OUTPUT

```
Test Case 1:
Shortest Distance: 7.0710678118654755
Shortest Path: [(1,2), (4,5), (7,1), (3,6), (1,2)]

Test Case 2:
Shortest Distance: 14.142135623730951
Shortest Path: [(2,4), (1,7), (6,3), (5,9), (8,1), (2,4)]
```

RESULT

The exhaustive search evaluated all permutations of the remaining cities and correctly identified the minimum-distance closed tour for both test cases, confirming the brute-force approach guarantees an optimal TSP solution for small instances.

Program 13: Assignment Problem using Exhaustive Search

AIM

To solve the assignment problem (assigning workers to tasks at minimum total cost) using exhaustive search over all possible worker-task permutations.

ALGORITHM

1. Read the n x n cost matrix, where cost[i][j] is the cost of assigning worker i to task j.
2. Define total_cost(assignment, cost_matrix) which sums cost[i][assignment[i]] for all workers i, for a given assignment (a permutation of task indices).
3. Generate all n! permutations of task indices (0..n-1), each permutation representing one possible complete assignment of workers to tasks.
4. For each permutation, compute its total cost using total_cost().
5. Track the minimum total cost seen and the permutation (assignment) that achieves it.
6. After all permutations are evaluated, print the optimal worker-task assignment and its total cost.

PROGRAM (C)

```
#include <stdio.h>

int n;
int cost[10][10];
int bestCost;
int bestAssign[10];

int totalCost(int assign[]) {
    int sum = 0;
    for (int i = 0; i < n; i++) sum += cost[i][assign[i]];
    return sum;
}

void permute(int arr[], int l, int r) {
    if (l == r) {
        int c = totalCost(arr);
        if (c < bestCost) {
            bestCost = c;
            for (int i = 0; i < n; i++) bestAssign[i] = arr[i];
        }
        return;
    }
    for (int i = l; i <= r; i++) {
        int t = arr[l]; arr[l] = arr[i]; arr[i] = t;
        permute(arr, l + 1, r);
        t = arr[l]; arr[l] = arr[i]; arr[i] = t;
    }
}

int main() {
    printf("Enter number of workers/tasks: ");
    scanf("%d", &n);
    printf("Enter cost matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    int tasks[10];
    for (int i = 0; i < n; i++) tasks[i] = i;
    bestCost = 1 << 30;
    permute(tasks, 0, n - 1);

    printf("Optimal Assignment: ");
    for (int i = 0; i < n; i++)
        printf("(worker %d, task %d) ", i + 1, bestAssign[i] + 1);
    printf("\nTotal Cost: %d\n", bestCost);
```

```
    return 0;
}
```

INPUT

```
Test 1: Cost Matrix = [[3,10,7],[8,5,12],[4,6,9]]
Test 2: Cost Matrix = [[15,9,4],[8,7,18],[6,12,11]]
```

OUTPUT

```
Test Case 1:
Optimal Assignment: [(worker 1, task 2), (worker 2, task 1), (worker 3, task 3)]
Total Cost: 19

Test Case 2:
Optimal Assignment: [(worker 1, task 3), (worker 2, task 1), (worker 3, task 2)]
Total Cost: 24
```

RESULT

The exhaustive search evaluated all n! possible worker-task assignments and correctly determined the minimum-cost assignment for both cost matrices.

Program 14: 0-1 Knapsack Problem using Exhaustive Search

AIM

To solve the 0-1 Knapsack problem — selecting items to maximize total value without exceeding a weight capacity — using exhaustive search over all possible subsets of items.

ALGORITHM

1. Read the number of items, their weights, values, and the knapsack capacity.
2. Define total_value(items, values) which sums the values of the selected item indices.
3. Define is_feasible(items, weights, capacity) which sums the weights of the selected items and checks that this sum does not exceed capacity.
4. Enumerate all 2^n subsets of items using a bitmask from 0 to 2^n - 1.
5. For each subset that is feasible (is_feasible returns true), compute its total value and compare with the best value found so far, updating the best subset if this one is higher.
6. After checking all subsets, print the optimal item selection and the maximum total value achieved.

PROGRAM (C)

```
#include <stdio.h>

int main() {
    int n;
    printf("Enter number of items: ");
    scanf("%d", &n);
    int weight[n], value[n];
    printf("Enter weights: ");
    for (int i = 0; i < n; i++) scanf("%d", &weight[i]);
    printf("Enter values: ");
    for (int i = 0; i < n; i++) scanf("%d", &value[i]);
    int capacity;
    printf("Enter capacity: ");
    scanf("%d", &capacity);
```

```
    int bestValue = 0, bestMask = 0;
    for (int mask = 0; mask < (1 << n); mask++) {
        int w = 0, v = 0;
        for (int i = 0; i < n; i++)
            if (mask & (1 << i)) { w += weight[i]; v += value[i]; }
        if (w <= capacity && v > bestValue) { bestValue = v; bestMask = mask; }
    }

    printf("Optimal Selection: [");
    int first = 1;
    for (int i = 0; i < n; i++)
        if (bestMask & (1 << i)) { printf(first ? "%d" : ",%d", i); first = 0; }
    printf("]\n");
    printf("Total Value: %d\n", bestValue);
    return 0;
}
```

INPUT

```
Test 1: Weights=[2,3,1], Values=[4,5,3], Capacity=4
Test 2: Weights=[1,2,3,4], Values=[2,4,6,3], Capacity=6
```

OUTPUT

```
Test Case 1:
Optimal Selection: [0, 2] (Items with indices 0 and 2)
Total Value: 7

Test Case 2:
Optimal Selection: [0, 1, 2] (Items with indices 0, 1, and 2)
Total Value: 10
```

RESULT

The exhaustive search examined all 2^n subsets and correctly identified the feasible subset with the maximum total value for both test cases, confirming an optimal solution for these small instances.

Program 15: Subset Sum Problem by Generating All Subsets

AIM

To solve the subset sum problem by exhaustively generating all possible subsets of a set and checking whether any subset's sum equals the target value.

ALGORITHM

1. Read the set of n integers and the target sum.
2. Represent each of the 2^n possible subsets using a bitmask from 0 to 2^n - 1.
3. For each bitmask, include element[i] in the subset if bit i is set, and compute the sum of the included elements.
4. If the computed sum equals the target sum, record and print this subset as the answer.
5. If no subset among all 2^n possibilities matches the target, report that no such subset exists.
6. Print the subset found and its sum.

PROGRAM (C)

```
#include <stdio.h>

int main() {
```

```
    int set[] = {15, 10, 12, 7, 5};
    int n = sizeof(set) / sizeof(set[0]);
    int target;
    printf("Enter target sum: ");
    scanf("%d", &target);

    int found = 0;
    for (int mask = 1; mask < (1 << n) && !found; mask++) {
        int sum = 0;
        for (int i = 0; i < n; i++)
            if (mask & (1 << i)) sum += set[i];

        if (sum == target) {
            printf("Subset found: [");
            int first = 1;
            for (int i = 0; i < n; i++) {
                if (mask & (1 << i)) {
                    if (!first) printf(",");
                    printf("%d", set[i]);
                    first = 0;
                }
            }
            printf("]\n");
            printf("Sum: %d\n", sum);
            found = 1;
        }
    }
    if (!found) printf("No subset with the given sum exists\n");
    return 0;
}
```

INPUT

```
Set: [15, 10, 12, 7, 5]
Target Sum: 22
```

OUTPUT

```
Subset found: [15,7]
Sum: 22
```

RESULT

The program generated all 2^5 = 32 possible subsets and correctly found the subset [15,7] whose elements sum to the target value 22.